

🔗 Chuyên đề: Cấu trúc Dữ liệu Các Tập hợp Rời nhau (Disjoint Set Union - DSU)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

🎯 1. Tổng quan & Bản đồ Tư duy (Mindmap)

Trong lý thuyết cấu trúc dữ liệu và lập trình thi đấu (Competitive Programming), **Cấu trúc Dữ liệu Các Tập hợp Rời nhau (Disjoint Set Union - DSU)** (còn gọi là **Union-Find**) là một trong những cấu trúc thanh lịch và đạt hiệu năng thực thi cao nhất.

DSU quản lý một phân hoạch của tập hợp gồm N phần tử $\{1, 2, \dots, N\}$ thành các tập hợp con không giao nhau, hỗ trợ 2 thao tác cơ bản:

1. `find(u)` : Xác định phần tử đại diện (gốc - root) của tập hợp chứa phần tử u . Hai phần tử u và v thuộc cùng một tập hợp khi và chỉ khi `find(u) == find(v)`.
2. `unite(u, v)` : Hợp nhất tập hợp chứa u và tập hợp chứa v thành một tập hợp duy nhất.

Nhờ sự kết hợp của hai kỹ thuật tối ưu hóa kinh điển: **Nén đường đi (Path Compression)** và **Hợp nhất theo kích thước / hạng (Union by Size / Rank)**, độ phức tạp thời gian cho mỗi thao tác đạt xấp xỉ hằng số:

$$O(\alpha(N))$$

trong đó $\alpha(N)$ là hàm nghịch đảo Ackermann ($\alpha(N) \leq 4$ với mọi $N \leq 10^{600}$, trên thực tế hoàn toàn tương đương $O(1)$).



⚡ 2. DSU Chuẩn mực: Hai Tối ưu hóa Nền tảng

2.1. Cấu trúc Cây Đại diện

Mỗi tập hợp được biểu diễn dưới dạng một cây có hướng, trong đó:

- Gốc của cây là **đại diện (representative)** của toàn bộ tập hợp.

- Gốc r tự trở vào chính nó: `parent[r] == r`.
- Ban đầu, mỗi phần tử là một tập hợp độc lập gồm 1 phần tử: `parent[i] = i`, `sz[i] = 1`.

2.2. Tối ưu 1: Nén đường đi (Path Compression)

Khi thực hiện `find(u)`, ta phải lần theo con trỏ `parent` để đi lên gốc r .

- **Ý tưởng:** Trên đường đi từ u lên gốc r , ta trở trực tiếp mọi đỉnh đi qua về gốc r .
- Lần sau khi gọi `find(u)` hoặc các đỉnh trung gian, khoảng cách lên gốc chỉ còn đúng **1 bước** ($O(1)$).

```
int find(int u) {
    if (u == parent[u]) return u;
    return parent[u] = find(parent[u]); // Nén đường đi thần tốc
}
```

2.3. Tối ưu 2: Hợp nhất theo Kích thước (Union by Size)

Khi gộp hai cây có gốc là $root_u$ và $root_v$:

- Nếu gộp ngẫu nhiên, cây có thể suy biến thành đường thẳng độ sâu $O(N)$.
- **Nguyên lý:** Luôn treo gốc của cây có **kích thước nhỏ hơn** vào gốc của cây có **kích thước lớn hơn**.
- Khi một đỉnh bị tăng độ sâu lên 1 đơn vị, kích thước cây chứa nó phải **tăng tối thiểu gấp đôi**. Do đó, độ sâu tối đa của cây không bao giờ vượt quá $\lfloor \log_2 N \rfloor$.

```
bool unite(int u, int v) {
    int root_u = find(u);
    int root_v = find(v);
    if (root_u == root_v) return false; // Đã cùng tập hợp

    // Treo cây nhỏ vào cây lớn
    if (sz[root_u] < sz[root_v]) swap(root_u, root_v);
    parent[root_v] = root_u;
    sz[root_u] += sz[root_v];
    return true;
}
```

🚩 Quan trọng:

****Định lý Phân tích Thời gian (Tarjan, 1975)**:**

- Chỉ dùng Union by Size: Mỗi thao tác mất $O(\log N)$.
- Chỉ dùng Path Compression: Chuỗi M thao tác mất $O(M \log N)$.
- ****Kết hợp cả hai**:** Mọi chuỗi M thao tác trên tập N phần tử có độ phức tạp trung bình là **** $O(M\alpha(N))$ **!**

🌀 3. DSU Thế vị / DSU Trọng số (Potentialized DSU)

Trong nhiều bài toán, ta không chỉ cần biết hai đỉnh u và v có cùng tập hợp hay không, mà còn cần biết **mối quan hệ tương đối** giữa chúng:

$$x_u - x_v = w \quad \text{hoặc} \quad \text{color}[u] \oplus \text{color}[v] = 1$$

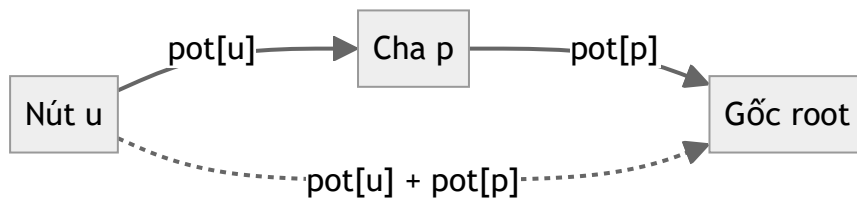
3.1. Cơ chế Lưu trữ & Cập nhật Vector Thế vị

Mỗi nút u lưu thêm một biến thế vị `pot[u]`:

$$\text{pot}[u] = \text{khoảng cách (hoặc hiệu đại số) từ } u \text{ đến cha trực tiếp } \text{parent}[u]$$

Khi thực hiện Nén đường đi trong `find(u)`:

```
int find(int u) {
    if (u == parent[u]) return u;
    int p = parent[u];
    parent[u] = find(p);
    pot[u] += pot[p]; // Cộng dồn thế vị dọc theo đường nén lên gốc!
    return parent[u];
}
```



3.2. Hợp nhất hai Phần tử với Ràng buộc $x_u - x_v = W$

Khi thêm một thông tin: $x_u - x_v = W$:

1. Tìm gốc $r_u = \text{find}(u)$ và $r_v = \text{find}(v)$.
2. Nếu $r_u == r_v$: Hệ đã xác định được quan hệ. Ta kiểm tra tính tương thích:

$$\text{pot}[u] - \text{pot}[v] \stackrel{?}{=} W$$

Nếu không bằng $\implies$ phát hiện **mâu thuẫn (Contradiction)**!

3. Nếu $r_u \neq r_v$: Ta gộp gốc r_u vào r_v . Để đảm bảo:

$$x_u - x_v = (\text{pot}[u] + x_{r_u}) - (\text{pot}[v] + x_{r_v}) = W$$

Ta suy ra giá trị thế vị cần gán cho cạnh mới nối từ r_u sang r_v :

$$\text{pot}[r_u] = W + \text{pot}[v] - \text{pot}[u]$$

3.3. Ứng dụng: Kiểm tra Đồ thị Hai phía Trực tuyến (Parity DSU)

- Bài toán: Cho các truy vấn thêm cạnh (u, v) lần lượt. Với mỗi cạnh, kiểm tra xem thêm vào có làm đồ thị xuất hiện chu trình lẻ (mất tính hai phía) hay không?
- Ta thay phép cộng số học bằng **phép XOR trên $\mathbb{F}_2$** :
- `pot[u] ∈ {0, 1}`: Tính cùng màu (0) hoặc khác màu (1) với gốc.
- Khi xét cạnh (u, v) : Cạnh yêu cầu u và v khác màu $\implies \text{pot}[u] \oplus \text{pot}[v] \equiv 1 \pmod{2}$.
- Nếu `find(u) == find(v)` và `(pot[u] ^ pot[v]) == 0` $\implies$ xuất hiện chu trình lẻ!

4. DSU Hoàn tác (Rollback DSU / Undo DSU)

Trong các bài toán động (Dynamic Graphs), các cạnh được thêm vào và **xóa đi theo thời gian**, hoặc ta cần quay lui trạng thái của DSU sau khi duyệt một nhánh tìm kiếm (Backtracking / Divide and Conquer over Time).

4.1. Quy tắc Sống còn: Cấm Tuyệt đối Path Compression!

🚨 Lưu ý đặc biệt:

****Quy tắc vàng của Rollback DSU**:**

- Khi sử dụng ****Nén đường đi (Path Compression)****, một lệnh `find` có thể thay đổi con trỏ cha của hàng chục đỉnh cùng lúc. Việc lưu lại toàn bộ các đỉnh bị thay đổi để hoàn tác sẽ làm phình to bộ nhớ và triệt tiêu tốc độ.
- ****Giải pháp bắt buộc****: ****CHỈ SỬ DỤNG Union by Size**** và ****KHÔNG dùng Path Compression****.
- Vì độ sâu của cây luôn bị chặn bởi $\lfloor \log_2 N \rfloor$, mỗi thao tác `find` mất chính xác **** $O(\log N)$ ****. Đổi lại, mỗi thao tác `unite` chỉ làm thay đổi đúng ****1 con trỏ cha**** và ****1 giá trị kích thước****, cho phép hoàn tác trong **** $O(1)$ ****!

4.2. Cài đặt Cấu trúc Rollback DSU Chuẩn mực

Ta duy trì một ngăn xếp `history` lưu lại các thông tin của lần gộp:

```

struct RollbackEvent {
    int u, v;          // Cây v được ghép vào u
    int sz_u;          // Kích thước ban đầu của u trước khi gộp
};

class RollbackDSU {
private:
    vector<int> parent, sz;
    vector<RollbackEvent> history;
    int num_components;

public:
    RollbackDSU(int n) : parent(n + 1), sz(n + 1, 1), num_components(n) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int u) const {
        while (u != parent[u]) u = parent[u]; // Không nên dùng đi!
        return u;
    }

    bool unite(int u, int v) {
        int root_u = find(u), root_v = find(v);
        if (root_u == root_v) {
            history.push_back({-1, -1, 0}); // Ghi nhận thao tác rỗng
            return false;
        }

        if (sz[root_u] < sz[root_v]) swap(root_u, root_v);

        history.push_back({root_u, root_v, sz[root_u]});
        parent[root_v] = root_u;
        sz[root_u] += sz[root_v];
        num_components--;
        return true;
    }

    void rollback() {
        assert(!history.empty());
        auto [u, v, old_sz] = history.back();
        history.pop_back();

        if (u == -1) return; // Thao tác không làm thay đổi cấu trúc

        parent[v] = v;      // Tách v ra khỏi u
        sz[u] = old_sz;     // Khôi phục kích thước cũ của u
        num_components++;
    }

    int snapshot() const {
        return history.size();
    }

    void rollback_to(int snap) {

```

```

while ((int)history.size() > snap) {
    rollback();
}
};

```

🌲 5. Cây Tái cấu trúc Kruskal (Kruskal Reconstruction Tree)

Bài toán: Cho đồ thị vô hướng N đỉnh có trọng số cạnh. Mỗi truy vấn (u, v) yêu cầu tìm đường đi nối u và v sao cho trọng số cạnh lớn nhất trên đường đi là nhỏ nhất có thể (Minimax Path).

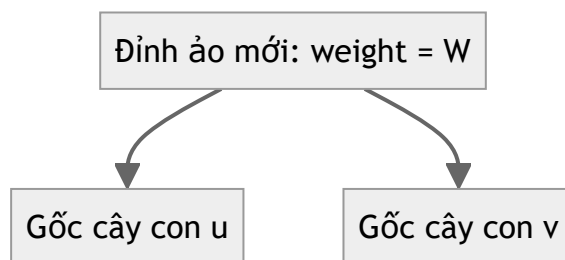
5.1. Cơ chế Dựng Cây Kruskal

Thay vì chỉ gộp u và v thông thường trong thuật toán Kruskal:

1. Ban đầu có N đỉnh gốc từ 1 đến N .
2. Sắp xếp các cạnh tăng dần theo trọng số.
3. Khi xét cạnh $e = (u, v)$ có trọng số W :
 - Nếu $root_u \neq root_v$, ta tạo ra một **đỉnh ảo mới** $node_{new} = ++N_{total}$.
 - Gán trọng số cho đỉnh ảo mới: $weight[node_{new}] = W$.
 - Nối cạnh từ đỉnh ảo mới tới $root_u$ và $root_v$:

$$parent[root_u] = node_{new}, \quad parent[root_v] = node_{new}$$

4. Sau khi duyệt hết cây khung, ta thu được một cây gồm $2N - 1$ đỉnh.



5.2. Tính chất Vàng

📌 Quan trọng:

****Định lý Kruskal Tree**:**

- Cây tái cấu trúc Kruskal là một ****Heap (Đống)** theo trọng số^{**}: Giá trị của đỉnh cha luôn lớn hơn hoặc bằng giá trị của mọi con cháu.
- Cạnh lớn nhất trên đường đi Minimax giữa hai đỉnh lá u và v trên đồ thị ban đầu chính là:

$$\text{Max Edge}(u, v) = weight[LCA(u, v)]$$

- Bài toán đường đi minimax trên đồ thị phức tạp được giải quyết trong $O(1)$ bằng truy vấn LCA trên cây Kruskal!

6. Bảng Trace Chi tiết: Parity DSU Kiểm tra Tính Hai phía

Xét 4 đỉnh $\{1, 2, 3, 4\}$. Lần lượt thêm các cạnh:

1. Thêm $(1, 2)$: Khác màu ($W = 1$).
2. Thêm $(2, 3)$: Khác màu ($W = 1$).
3. Thêm $(3, 4)$: Khác màu ($W = 1$).
4. Thêm $(1, 4)$: Khác màu ($W = 1$).

Diễn biến từng bước:

Bước	Thao tác cạnh	$find(u)$ & $find(v)$	Quan hệ hiện tại	Hành động DSU	Kết luận
1	$(1, 2, W = 1)$	$r_1 = 1, r_2 = 2$	Chưa cùng tập	Gộp 2 vào 1, $pot[2] = 1 \oplus 0 \oplus 0 = 1$	Hợp lệ (1 và 2 khác màu)
2	$(2, 3, W = 1)$	$r_2 = 1, r_3 = 3$	Chưa cùng tập	Gộp 3 vào 1, $pot[3] = 1 \oplus pot[2] = 1 \oplus 1 = 0$	Hợp lệ (3 cùng màu 1, khác 2)
3	$(3, 4, W = 1)$	$r_3 = 1, r_4 = 4$	Chưa cùng tập	Gộp 4 vào 1, $pot[4] = 1 \oplus pot[3] = 1 \oplus 0 = 1$	Hợp lệ (4 cùng màu 2, khác 3)
4	$(1, 4, W = 1)$	$r_1 = 1, r_4 = 1$	Đã cùng tập!	Kiểm tra: $pot[1] \oplus pot[4] = 0 \oplus 1 = 1$	Khớp với $W = 1 \implies$ Chu trình độ dài 4 chẵn, đồ thị vẫn là hai phía!

Giả sử bước 4 yêu cầu cạnh $(1, 3)$ khác màu ($W = 1$):

- $r_1 = 1, r_3 = 1$. Ta có $pot[1] \oplus pot[3] = 0 \oplus 0 = 0$ (nghĩa là 1 và 3 phải cùng màu).
- Yêu cầu khác màu mâu thuẫn với thực tế $\implies$ xuất hiện chu trình lẻ $(1 - 2 - 3 - 1)$ độ dài 3 $\implies$ Đồ thị không còn hai phía!

7. Cạm bẫy Phòng thi & Kỹ thuật Tối ưu

1. **Nhầm lẫn giữa `u` và `find(u)`** :
2. Trong hàm `unite(u, v)`, nếu viết `parent[u] = v` thay vì `parent[root_u] = root_v`, cấu trúc cây sẽ bị gãy vụn và dẫn đến chu trình vô hạn trong hàm `find`.
3. **Quên khởi tạo DSU**:
4. Luôn đảm bảo mảng `parent` được gán `parent[i] = i` và `sz[i] = 1` với mọi $i \in [1, N]$. Với bài toán có nhiều test case (T test), quên khởi tạo lại DSU sẽ gây ra **Wrong Answer** hàng loạt.

5. **Dùng Path Compression trong Rollback DSU:**

6. Khi cài đặt bài toán Đồ thị động (Dynamic Connectivity) hoặc Chia để trị, việc vô tình để dòng `parent[u] = find(parent[u])` sẽ phá hủy toàn bộ khả năng khôi phục của mảng `history`, dẫn đến lỗi bộ nhớ hoặc tính toán sai lệch hoàn toàn.

7. **Không xử lý mảng kích thước $2N$ trong Kruskal Reconstruction Tree:**

8. Cây Kruskal tạo thêm $N - 1$ đỉnh ảo mới $\implies$ tổng số đỉnh lên tới $2N - 1$. Mọi mảng lưu trữ (`parent`, `depth`, `adj`, Sparse Table) đều phải khai báo kích thước tối thiểu $2 \cdot MAXN$.

8. Tài liệu Tham khảo & Đọc thêm

1. **CP-Algorithms:**

2. [Disjoint Set Union \(DSU\)](#) — Phân tích hàm Ackerman và kỹ thuật gộp theo kích thước.

3. **VNOI Wiki:**

4. [Cấu trúc các tập hợp rời nhau](#) — DSU thế vị và kiểm tra đồ thị hai phía.

5. [DSU có hoàn tác \(DSU with Rollback\)](#) — Ứng dụng trong đồ thị động và chia để trị thời gian.

6. **USACO Guide (Gold Section):**

7. [Disjoint Set Union](#) — Cây tái cấu trúc Kruskal và liên thông cạnh.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.